InferX

单卡大模型推理部署与优化

陈文欣、张志雄、郭坤昌

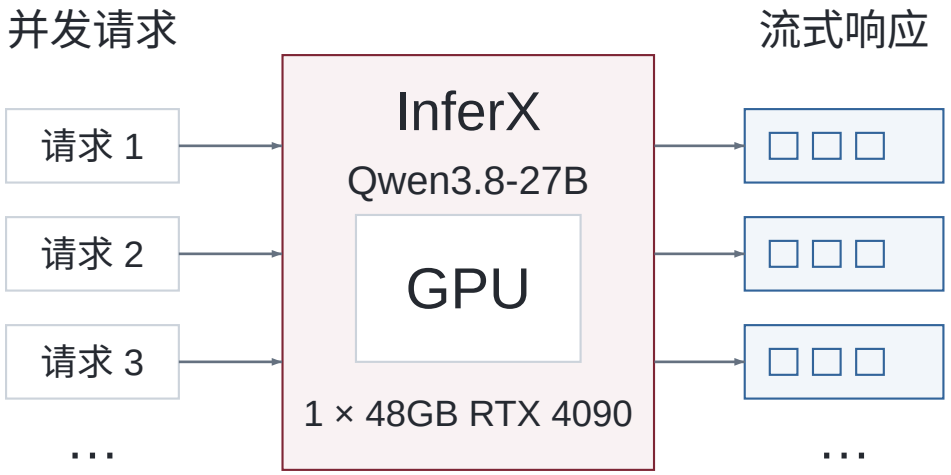
你说啥都队

项目介绍与答辩 · 赛道 A · 2026-10-18

github.com/Yuer-yuan/InferX

项目概述

目标：在 SLO 约束下，提高单卡并发能力。



首字与生成延迟均受 SLO 约束

TTFT P95 ≤ 10s · TPOT P95 ≤ 40ms

交付：基于 NInfer 的推理服务

1 **4bit 计算**

权重与激活均采用 INT4 矩阵运算。

2 **MTP 并发适配**

扩展批处理能力，各请求独立提交、回滚状态。

3 **基于请求进度的调度**

长输入分块处理；按进度和紧迫性安排解码。

结果： 支持 14 并发请求，两次测试均满足 SLO；总吞吐 306.04–309.95 tok/s。

精度： IFBench 前 100 题：严格通过 85 题，宽松通过 88 题。

问题与目标：影响并发能力的两项瓶颈

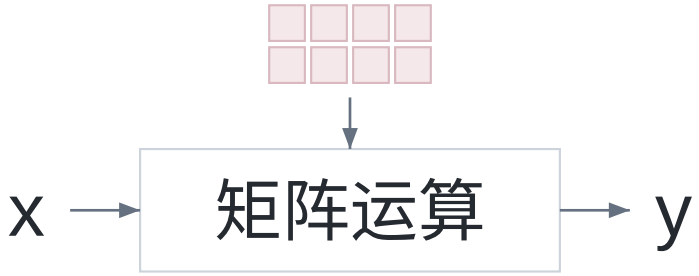
I 预填充与解码干扰



解码等待

共享 GPU 的执行时间

II 算子效率

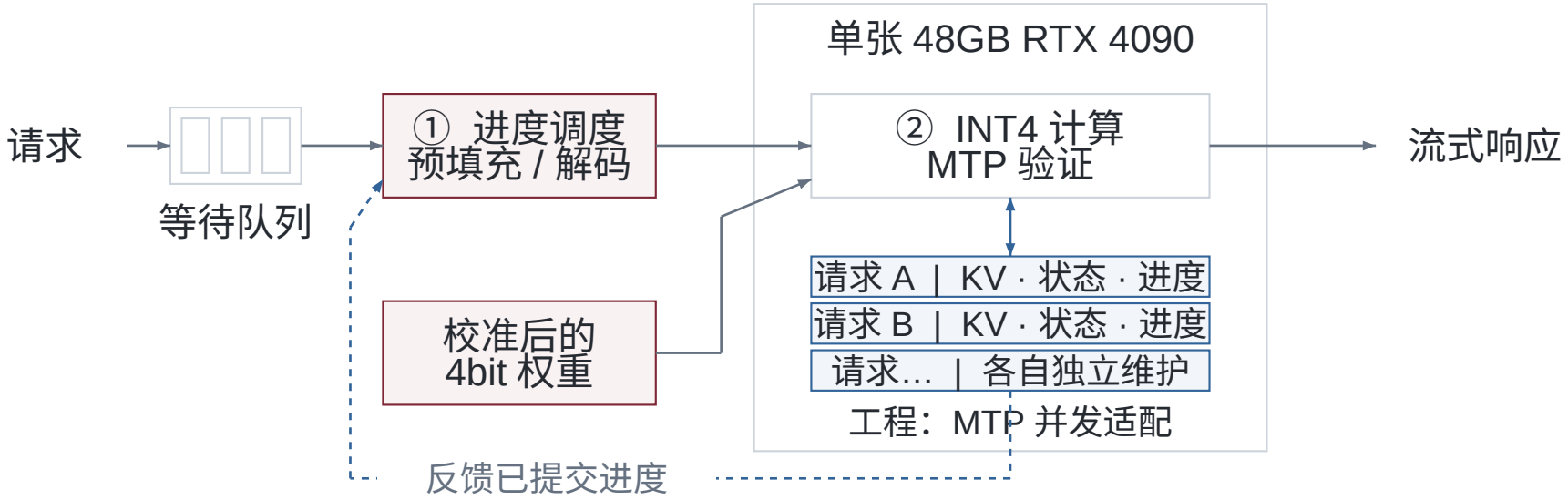


计算与权重访存开销

- **I · 预填充与解码相互干扰。** 长输入的预填充（Prefill）占用 GPU，延后解码（Decode），增加尾延迟。
- **II · 算子效率。** 长输入处理与批量验证的矩阵形状不同；低比特量化的收益受算子、数据转换和访存开销影响。

目标：提高满足 TTFT P95 ≤ 10s、TPOT P95 ≤ 40ms 的并发能力，并验证任务质量。

整体方案：调度与 INT4 优化，MTP 并发适配



技术路线

评测模型：Qwen3.8-27B

- **离线：** GPTQ 校准与权重量化。
- **在线：** 分块调度 → INT4 / MTP；
  按请求提交状态、反馈进度。

框架取舍：硬件适配与改造成本

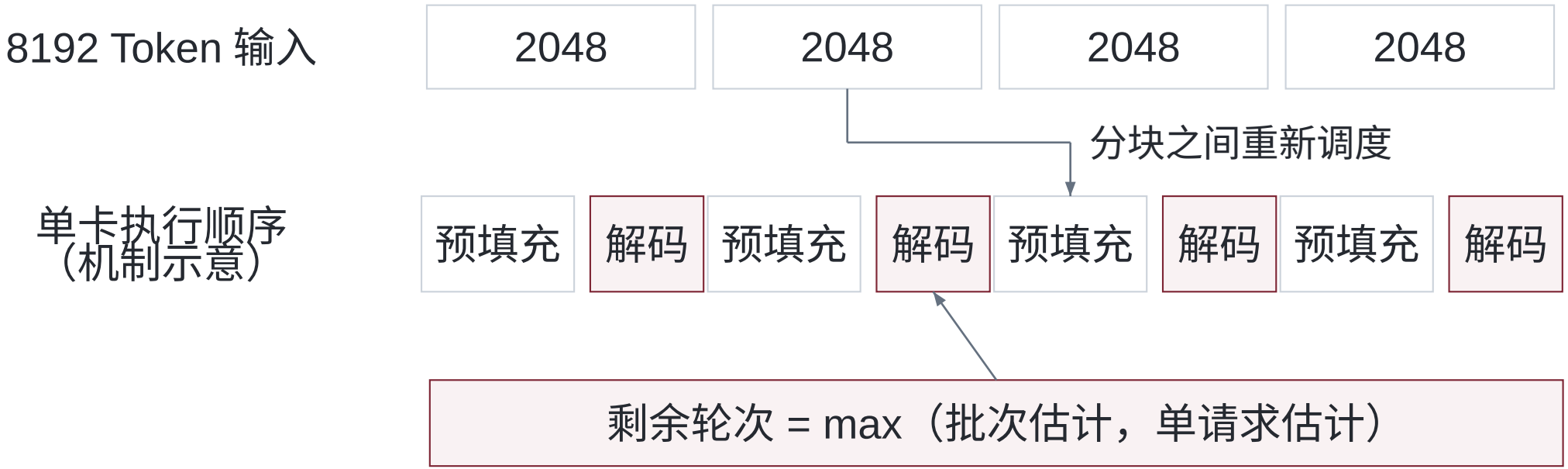
框架	依据与取舍
NInfer（采用）	面向 4090 / SM89，复用量化与 MTP； 统一改造算子、状态和调度，需自行维护。
vLLM / SGLang	支持 MTP，适配取决于具体模型； 本项目的量化与状态管理改造需额外适配。

选型依据：ninfer-4090 的针对性优势

相较 vLLM / SGLang，更贴合本项目的固定模型、单张 4090 与底层改造。

优势	ninfer-4090 的基础	对本项目的价值
模型与硬件专用适配	面向 Qwen3.8-27B 与单张 RTX 4090；采用 SM89 原生 CUDA 执行路径。	围绕固定模型结构与矩阵形状优化，聚焦 4090 上的单卡推理。
低位执行链直接复用	Q4/Q5 权重、量化 KV 与 MTP 已配套；目标模型的加载与验证路径可复用。	在已有部署链上接入 INT4 计算，减少定制路径的迁移与集成工作。
执行与状态统一控制	C++20 / CUDA 直接管理执行；ReplaySSM 提供状态提交与回滚。	便于联改算子、批验证与调度，扩展状态、工作区与 CUDA Graph。

设计 I：分块预填充与请求进度调度



- **调度时机：** 8192 Token 输入拆为 4 个 2048 块；每块结束后重新选择预填充或解码，块内不抢占。
- **剩余工作：** 按请求统计已提交 Token 和解码轮数，估计每轮产出；用批次均值会低估慢请求的剩余轮次。
- **决策依据：** 剩余轮次乘以当前批次轮耗时，得到预计解码耗时；比较两阶段时限余量，保留预填充期限约束。

设计 I：从实际提交进度估计剩余轮次

每个请求独立估计

$$y_i = \frac{g_i - 1}{d_i}$$

$$R_i = \max\left(\left\lceil \frac{r_i}{\max(1, y_b)} \right\rceil, \left\lceil \frac{r_i}{\max(1, y_i)} \right\rceil\right)$$

g：已提交 Token 数；d：解码轮数
r：剩余 Token 预算；y：每轮产出
i：单个请求；b：当前批次
减 1 排除预填充产生的首个 Token。

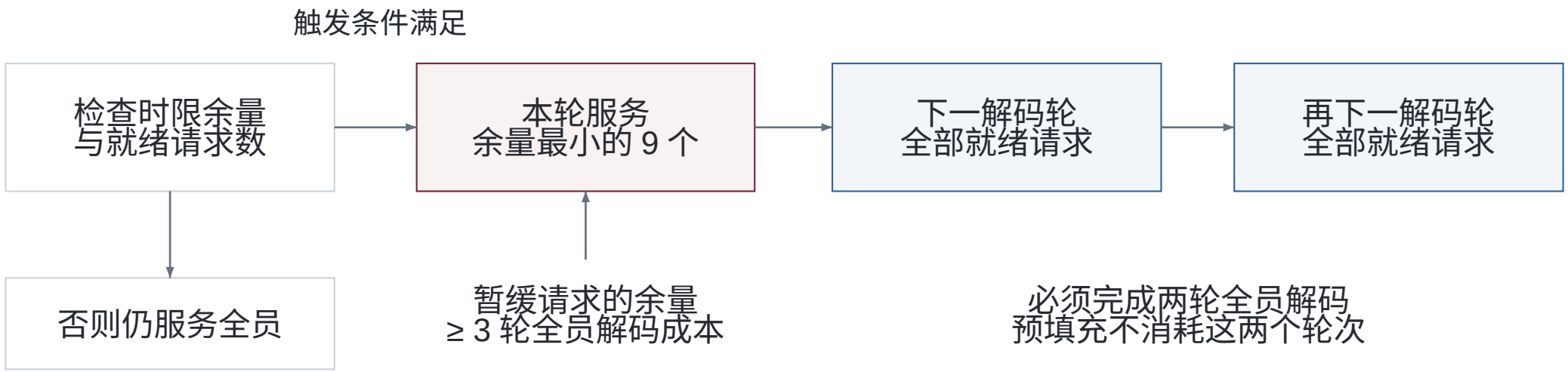
- **提交后更新。** 统计经输出裁决的实际提交量；取消行不计入解码轮数。
- **无历史时回退。** 尚无解码记录时，沿用当前批次估计。
- **保留原估计下限。** 慢请求可以提高剩余轮次，快请求不压低原估计。

同机 16 并发，仅改进度估计：128 条正式请求 + 8 条预热

估计方法	TTFT P95 (s)	TPOT P95 (ms)	tok/s
批次估计	9.856	48.784	311.77
加入请求进度	9.838	45.745	315.73

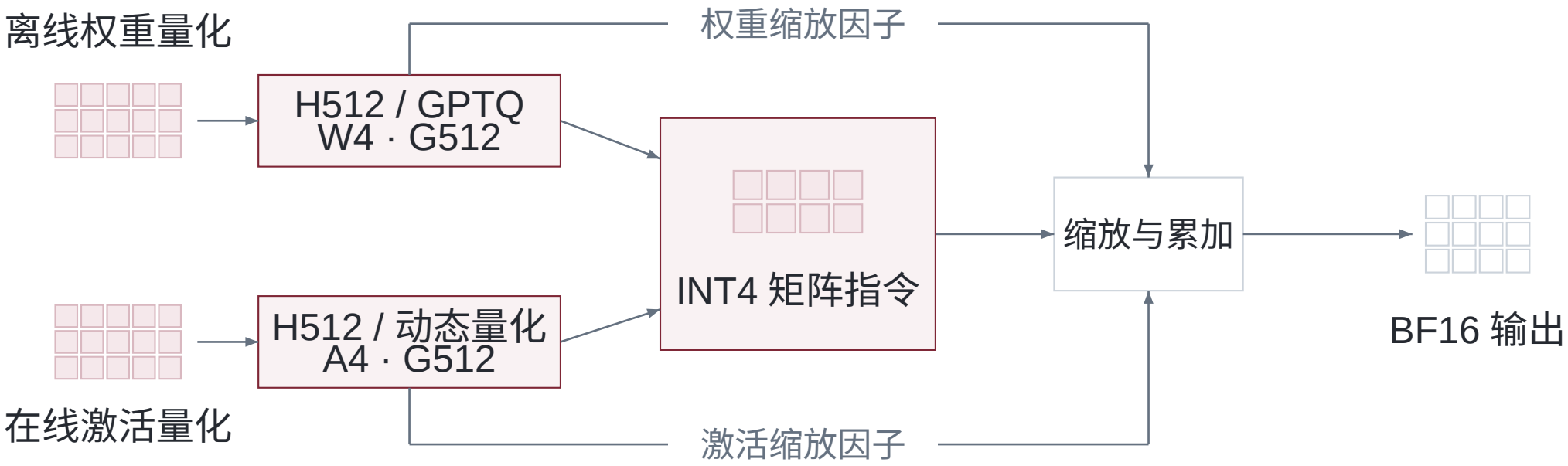
TPOT 改善 6.23%，仍高于 40ms；模型算子与 CUDA Graph 保持一致。

设计 I：有界选择紧迫请求，随后恢复全员服务



- 先由原调度器选择解码；仅当 10–14 个请求就绪、存在负时限余量且暂缓请求余量达标时缩减。
- 每次最多连续缩减一轮；随后两个实际解码轮服务全部就绪请求。预填充不消耗这两个轮次。
- 仅给实际服务的请求记解码进度；这是服务机会的重分配，总吞吐和 SLO 效果需用完整服务实测。

设计 II：权重与激活的 INT4 计算



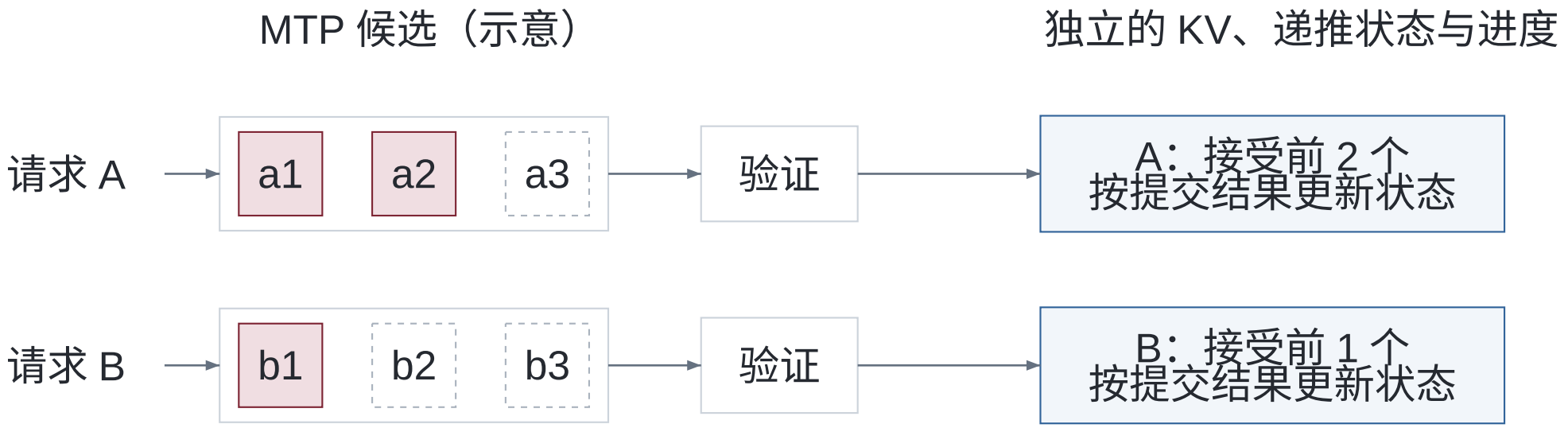
- **离线权重：** 每 512 维做局部 Hadamard 变换，按 512 通道分组做 GPTQ 校准；缓存 4bit 码与缩放因子。
- **在线激活：** BF16 激活做对应变换与逐组动态量化；权重、激活均送入 S4 整数矩阵指令。
- **输出恢复：** 组内 INT32 累加，组间结合缩放因子做 FP32 累加，输出 BF16；保留非线性和残差的原有语义。

设计 II：从量化缓存接入真实模型

改造位置	具体实现	验证重点
量化缓存	先覆盖 64 层 FFN 的 128 个矩阵；校准用 4 条 8192 Token 请求，另留 2 条验证。	核对码值与缩放因子，检查留出数据误差。
算子与路由	从 FFN 扩展至输入/输出投影；接入真实权重、工作区和 Graph 依赖。	完整计时包括变换、量化、矩阵计算与写回。
MTP 验证阶段	47 层 GDN 输出投影启用 T33–128；T1–32、L0 与全注意力输出保留原路由。	T 为矩阵 Token 行数；按层、阶段和形状核验覆盖。

- **本项目实现：** 基于已有 GPTQ 与 Hadamard 方法，完成缓存生成、CUDA 算子及模型路由适配。

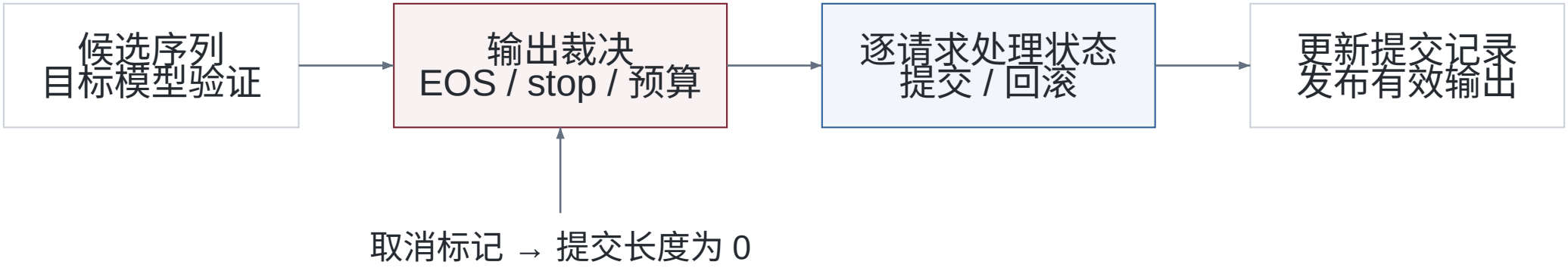
工程实现：MTP 的并发适配



实线框：接受；虚线框：拒绝。省略回退 Token，接受数与最终提交数不同。

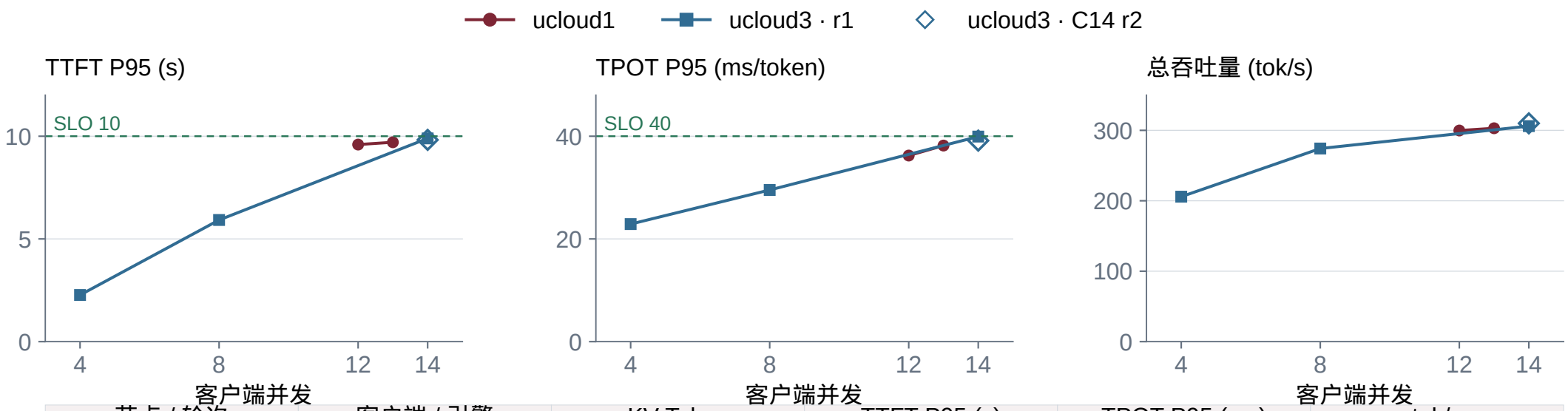
- **算子与布局：** 扩展 attention/GDN 验证形状、行到槽位映射、输入输出缓冲和状态记录，支持 16/32 请求。
- **运行时资源：** 按并发配置分配 KV、状态和工作区，同步更新 CUDA Graph 捕获参数及缓冲区边界。
- **一致性要求：** 各请求独立维护 KV、GDN 递推及卷积状态；接受数量可不同，状态必须对应实际提交序列。

工程实现：按请求提交与回滚状态



- **先裁决，再提交：** 目标验证后，按 EOS/stop 和剩余预算裁定提交长度；取消请求不提交。
- **按请求更新：** 先处理待提交状态，再更新已提交 Token、生成预算和解码轮数；最后发布有效输出。
- **验证覆盖：** 检查高编号槽位、0–4 部分提交、取消/重置与 Graph 重放。状态融合阶段 2013 条记录一致，86 项测试通过、4 项跳过。

结果与证据：并发请求下的延迟与吞吐



节点 / 轮次	客户端 / 引擎	KV Token	TTFT P95 (s)	TPOT P95 (ms)	tok/s
ucloud3 · r1	4 / 4	65536	2.269	22.891	205.82
ucloud3 · r1	8 / 8	69632	5.922	29.532	274.20
ucloud1 · r1	12 / 12	104448	9.594	36.235	299.76
ucloud1 · r1	13 / 13	113152	9.706	38.181	302.97
ucloud3 · r1	14 / 13	121856	9.894	39.929	306.04
ucloud3 · r2	14 / 13	121856	9.822	39.142	309.95

每次输入 8192 Token、输出 512 Token；每配置 128 次测试、8 次预热。

每条曲线对应一台节点；14 并发测了两次，等待时间计入首字延迟。

结果与证据：达标配置与失败边界

14 并发两次测试均达标；15、16 并发的生成延迟仍超限。

节点 / 轮次	客户端 / 引擎	TTFT P95 (s)	TPOT P95 (ms)	tok/s	SLO
ucloud1 · r1	14 / 14	9.725	41.022	306.06	FAIL
ucloud3 · r1	14 / 13	9.894	39.929	306.04	PASS
ucloud3 · r2	14 / 13	9.822	39.142	309.95	PASS
ucloud1 · r1	15 / 13	9.96	41.881	309.66	FAIL
ucloud3 · r1	16 / 16	9.967	51.221	316.74	FAIL

- 达标配置：引擎最多接纳 13 个活动请求，其余请求等待；等待计入首字延迟。
- 目标：首字延迟 P95 ≤ 10s，逐 Token 生成延迟 P95 ≤ 40ms。

结果与证据：IFBench 前 100 题的指令遵循

前 100 道题，共 111 条指令约束

评分对象	结果
题目严格评分	85 / 100
题目宽松评分	88 / 100
指令严格评分	96 / 111
指令宽松评分	99 / 111

- 评测范围：原评测脚本选取的 IFBench 前 100 题，按原评分器自动判定。
- 题目严格评分：一道题的全部指令约束通过，才算该题通过。
- 测试设置：开启思考模式，最多生成 32768 Token；与性能测试使用同一二进制。

质量案例：成功与失败的具体回答

IFBench 前 100 题中的实际回答

成功 · key80

题目要求

以 Arya 将别人伪装成自己为主题，写三句等长、用词不重复的句子；每行逐级缩进。

实际回答（完整）

```
She swapped faces with ease.
    Her mimicry took all shapes.
        Every mask looked like hers.
```

- 缩进为 0 / 4 / 8 个空格；去除缩进后，每句均为 28 个字符，15 个单词无重复。
- 两条指令均通过，题目严格评分为 1。

失败 · key88

题目要求

回答给定流动问题的 n 范围；以 !?!? 列表作答，相邻单词的首字母不能相同。

实际回答（完整）

```
!?? 0 < n < 1
!?? valid range
```

- 两项都用 !??，要求为 !?!?；标记少一个字符。
- 两条指令仅一条通过，因此这道题未通过。

工程化设计：获取与校验

① 取得复验材料，在新目录准备输入

```
export INFERX_REPRO=/workspace/InferX/reproductions/rerun01
mkdir -p /workspace/InferX/reproductions
mkdir "$INFERX_REPRO"
tar -xzf reproduction-kit-v1.tar.gz \
  -C "$INFERX_REPRO" --strip-components=1
cd "$INFERX_REPRO"
sha256sum -c SHA256SUMS
```

输入	固定身份	本轮核验
完整引擎源码	1023 个文件 · c976692d…	归档及逐文件 SHA；构建前后复核
模型与量化缓存	固定模型 revision；025 / 041 / 047	模型及 478 个服务文件重新计算 SHA

在复验材料目录运行；u3 源码复验使用原只读缓存；u2 量化重建的新产物独立验证。完整流程见 reproduction.md。

工程化设计：量化缓存重建

② 从 FP8 与固定校准输入重建三类缓存

```
python3 "$INFERX_Q/reproduce.py" \
  --work-dir "$INFERX_W" --fp8 "$INFERX_FP8" \
  --model "$INFERX_M" --tokenizer "$INFERX_T" \
  --fit-python "$FIT_PY" --build-env "$INFERX_B" \
  --cuda-root "$INFERX_C" --xgrammar "$INFERX_X" \
  --eval-envs "$INFERX_E" --tools "$INFERX_U"
```

阶段	重新采集与拟合	生成对象
025	原 Q4/Q5 前向采集 → FFN 拟合	64 层 gate/down · 256 文件
041	加载新 025 采集 → 输入投影拟合	64 层输入投影 · 128 文件
047	加载新 025＋041 采集 → 输出投影拟合	固定 47 层输出投影 · 94 文件

固定 4 条训练、2 条留出输入；FP8 按逐文件 revision/SHA 获取，基础.ninfer 模型单独获取。

路径变量见复验指南；116 新量化阶段与最终回执见独立附件。以下 664 复验使用旧缓存。

工程化设计：源码构建

③ 正常完整构建，绑定新二进制

```
python3 build_owner.py
python3 bind_rebuilt_recipe.py
python3 -B -m unittest discover \
  -s entrypoint/tests -p "test_*.py" -v
```

- Release / SM89 · CUDA13.2.51 / GCC13.4；新 build 目录完整编译。
- 构建前后 1023 个源文件一致；7 项调度 CPU 测试通过，保存 55 项动态依赖。
- 构建产物与运行配置一起归档；后续性能、精度复验直接使用这一构建。

核对 build-job.json、runtime-identity.json 和 rebuilt-recipe-binding.json。

工程化设计：启动服务

④ 在新部署目录启动性能服务

```
python3 -B entrypoint/serve_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --profile perf-c14 --root "$PWD" --port 18540 \
  --log-dir "$PWD/service-perf-c14-001"
```

- 客户端 14 / 引擎 13；普通 MTP K3，KV121856，上下文 65536，输入块 2048。
- 本次服务和客户端均用新构建配方；19 个环境开关保持冻结 100 配置。
- 服务日志和 launch.json 记录真实进程、二进制、路径与参数。

工程化设计：性能复验

⑤ 同机另一个终端运行原性能评测

```
OPENAI_API_KEY=EMPTY python3 -B entrypoint/perf_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --concurrency 14 --root "$PWD" \
  --service-launch "$PWD/service-perf-c14-001/launch.json" \
  --base-url http://127.0.0.1:18540/v1 \
  --output-dir "$PWD/performance-C14-001"
cat performance-C14-001/summary.json
```

- 128 正式＋8 预热＋1 探测；冷 8192 / 输出 512，保留正式阶段初始请求。
- 入口核对真实服务身份和全部 137 条请求；原始记录在 performance-C14-001/c14。

工程化设计：复验性能结果

⑥ 本次复验的原始 JSON 节选

ucloud3 · 新二进制 664cfbb9… · 客户端 14 / 引擎 13 · 延迟单位 ms


```
{
  "concurrency": 14,
  "records": 128,
  "error_requests": 0,
  "metrics": {
    "time_to_first_token": {"p95": 9896.41735465},
    "inter_token_latency": {"p95": 38.2858495440313}
  },
  "output_token_throughput": 304.89665937584215
}
```


TTFT 9.896s、TPOT 38.286ms：本轮两项 SLO 通过；吞吐 304.90 tok/s。

工程化设计：质量复验

⑦ 正常停止性能服务，再启动质量服务

```
python3 -B entrypoint/serve_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --profile quality100 --root "$PWD" --port 18541 \
  --log-dir "$PWD/service-quality100-002"
```

同机另一个终端评测

```
OPENAI_API_KEY=EMPTY python3 -B entrypoint/accuracy_candidate.py \
  --recipe entrypoint/candidate_recipes/100-k3-u3-47-rebuilt-20261018.json \
  --root "$PWD" \
  --service-launch "$PWD/service-quality100-002/launch.json" \
  --base-url http://127.0.0.1:18541/v1 \
  --output-dir "$PWD/quality100-002"
```

客户端/引擎 13 · KV532480 · max32768 · 自然 EOS、默认 thinking 与采样。

入口先核对进程并等待模型就绪；原始质量输出在 quality100-002/evaluation/accuracy。

⑧ IFBench 前 100 题的复验评分

新二进制 664cfbb9… · C13 / KV532480 · 分值为比例


```
{
  "accuracy": {
    "status": "完整",
    "generated_samples": 100,
    "metrics": {
      "prompt_level_strict": {"score": 0.9, "num": 100},
      "inst_level_strict": {"score": 0.9009, "num": 111},
      "prompt_level_loose": {"score": 0.96, "num": 100},
      "inst_level_loose": {"score": 0.955, "num": 111}
    }
  }
}
```

工程化设计：异常处理

异常场景	处理方式	保留记录
启动或端口故障	启动前检查端口，设置服务就绪超时。	启动日志、退出码、实际配置。
显存不足	检查 KV 缓存、并发上限和 CUDA Graph 占用，调整后复测。	OOM 日志与原参数。
非法请求	模型 ID 或输入 Token 无效时，返回 HTTP 400。	请求、状态码与错误响应。
请求异常结束	保留超时、空输出及截断记录，按原规则统计。	客户端记录、结束原因、输出长度。

工程化设计：可靠性

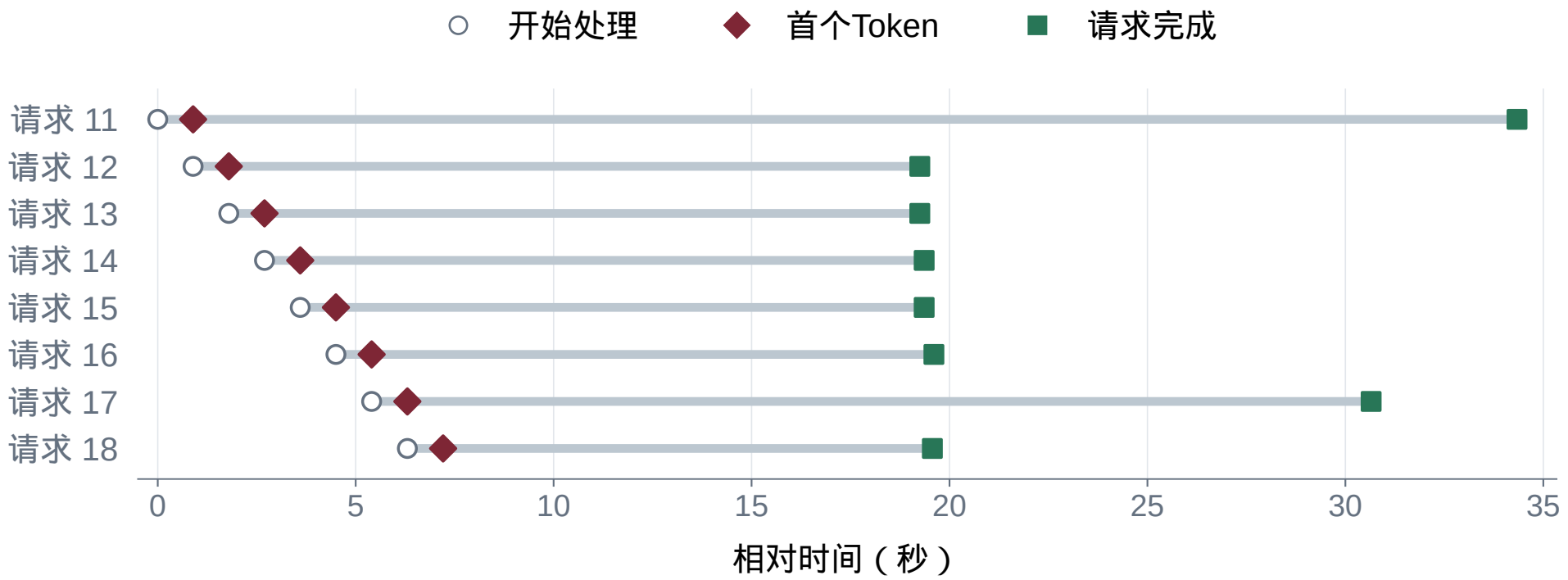
检查对象	一致性要求	验证方式
API 输出	流式响应的结束状态、Token 计数与结束原因一致。	通过真实请求验证流式与非流式响应、用量统计及参数校验。
并发状态	各请求独立提交、回滚，复用槽位时保持状态隔离。	覆盖高编号槽位、部分提交、取消/重置及 CUDA Graph 重放。
服务进程	清理前核对身份，避免误停其他进程。	核对 PID、启动时间及可执行文件，通过 pidfd 回收进程。

状态融合实验：2013 条配对记录一致；86 项测试通过，4 项跳过。

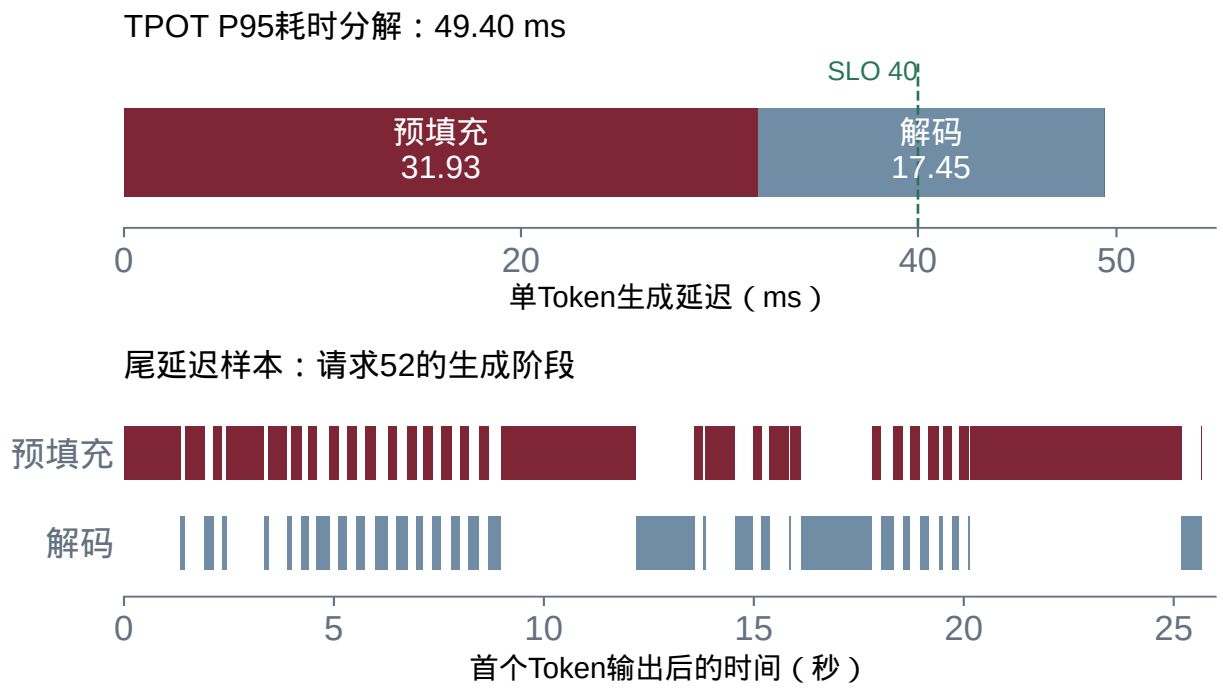
工程化设计：可观测性

请求执行时序：开始处理 → 首个 Token → 完成

085 版本、16 并发：展示最先开始处理的 8 个正式请求，以首个请求开始处理为零点

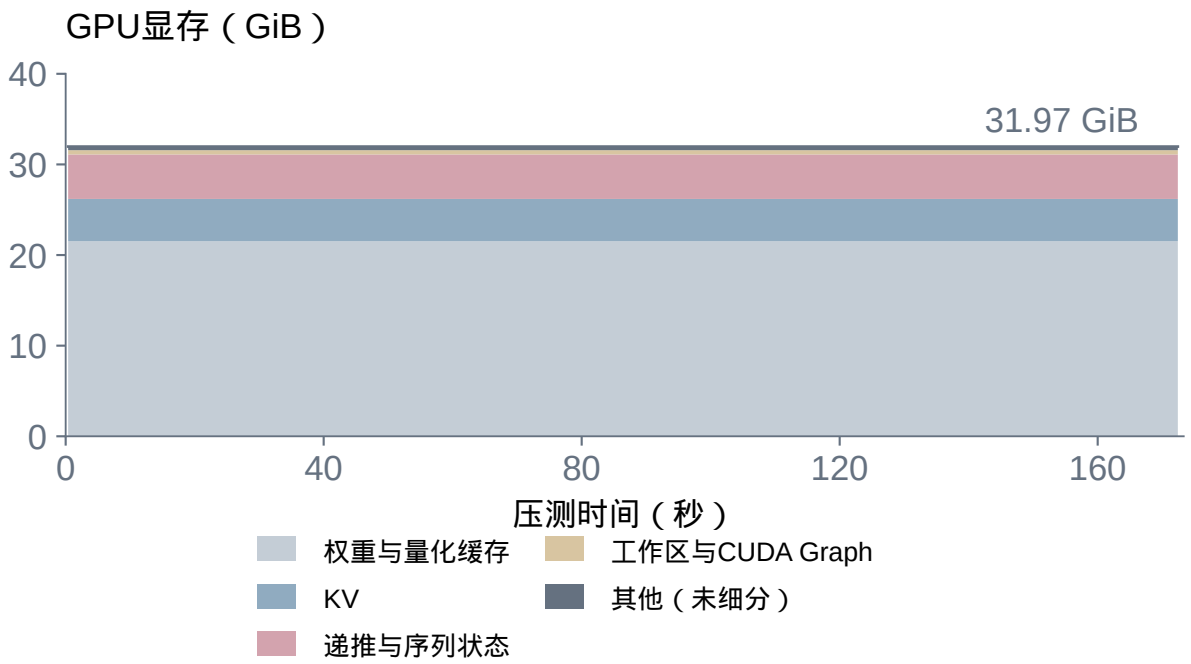


当前瓶颈：预填充干扰解码执行



- TPOT P95 为 49.40ms，其中其他请求的预填充占 31.93ms（64.6%），本请求参与的解码占 17.45ms。
- 8192 Token 输入的预填充耗时中位数为 0.909s，仍是优化重点。
- 085 版本 16 并发单轮诊断，128 个请求、无前缀缓存复用；按 P95 对应请求加权分解。
- 依据执行器同步记录的耗时分析请求间干扰，不代表 GPU 独占执行时间。

资源消耗：推理服务的显存占用



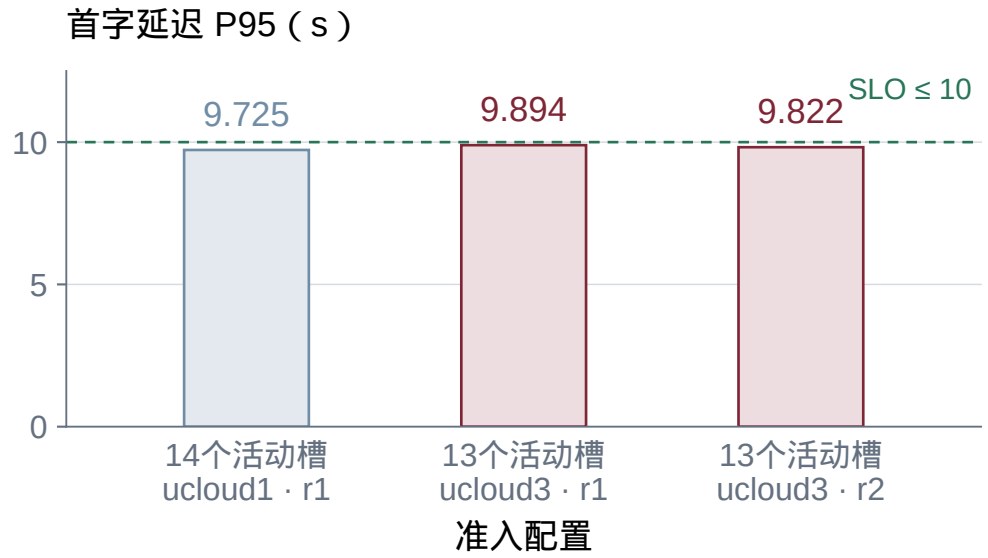
显存组成（GiB）

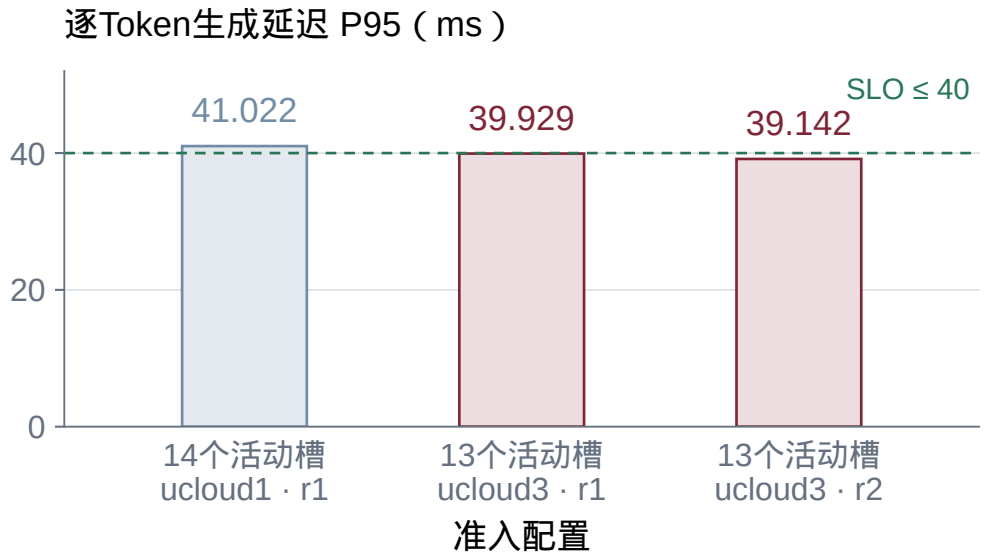
权重与量化缓存	21.53
KV 缓存	4.66
递推与序列状态等	4.91
工作区与 CUDA Graph	0.47
其他（未细分）	0.40

- 085 同轮测试采样 173 秒、共 173 次，显存占用均为 31.97GiB。
- 分项依据启动分配记录，总量依据设备采样；CUDA Graph 实际占用 138MiB。

并发取舍：用首字延迟预算缓解生成阶段竞争

同为 14 并发请求，比较引擎的两种活动请求上限。





- 机制与代价：最多接纳 13 个活动请求，其余等待；等待计入 TTFT，以此限制生成阶段的资源竞争。
- 选择依据：14 个活动槽的 TPOT 超限；13 个槽两轮同时达标，总吞吐 306.04–309.95 tok/s。

总结与不足

INT4 执行 → **请求状态隔离与调度** → 延迟约束下的并发能力

- 完成 INT4 推理、进度调度与 MTP 并发适配；14 并发两次测试均满足 SLO。
- IFBench 前 100 题：严格通过 85 题，宽松通过 88 题。
- 当前不足：预填充仍影响解码，15/16 并发的生成延迟超限。
- 下一步：优化预填充算子与调度，增加重复测试和完整数据集评测。